

# 95. Quatre nuances, deux bits par pixel

Sur la Game Boy d'origine, un pixel n'est pas une couleur : c'est un nombre de 0 à 3.

## 1. Ce qu'il faut comprendre

L'écran de la Game Boy d'origine ne connaît que **quatre nuances** : du plus clair au plus sombre. Tout ce que tu as vu dans les jeux — Tetris, Zelda, Pokémon Rouge — est fait avec ces quatre-là, et rien d'autre.

Quatre valeurs, c'est **deux bits** par pixel (00, 01, 10, 11 : 0, 1, 2, 3). Une tuile de  $8 \times 8$  pixels pèse donc  $64 \times 2 = 128$  bits = **16 octets**. La console range ces deux bits dans deux octets séparés par rangée : le premier porte le bit du bas de chaque pixel, le second le bit du haut. Le compilateur fait ce rangement pour toi.

Dans le programme, une tuile s'écrit en **huit rangées de huit caractères**. Deux alphabets, au choix : les **chiffres** `0 1 2 3`, ou les **signes** `. - + #` — le point est le plus clair, le dièse le plus sombre. Les signes se lisent mieux d'un coup d'œil ; les chiffres disent exactement le nombre rangé.

Retiens le mot juste : le 0-3 écrit dans la tuile est un **indice**, pas encore une nuance. C'est la **palette** qui décide quelle nuance l'écran montre pour chaque indice. D'origine, la palette est « 0 → 0, 1 → 1, 2 → 2, 3 → 3 », et l'on ne voit pas la différence. La leçon d'après la fera apparaître.

**À toi** : dessine une tuile en damier (une case sur deux en 0 et en 3), et pose-la sur toute une ligne.

### Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <ecran>` → [27.1. La fonction ecran\(\) — éteindre et rallumer l'écran](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **504 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         94 o ← démarrage - ta fonction
VBlank          8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage   28 o ← image() - attend l'image suivante
AttendreVBlank  16 o ← ecran() - attend que l'écran soit libre
AttendreAcces   12 o ← EcrireTexte - attend de pouvoir écrire une case
AdresseCase     10 o ← poser() - trouve l'adresse d'une case de l'écran
EcrireTexte     10 o ← texte() - écrit un texte, lettre par lettre
CopierTuiles    67 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         176 o ← CopierTuiles - 9 lettres sur 43, et 2 tuiles dessinées
"EN CHIFFRES"   11 o ← texte() - un texte gravé
"EN SIGNES"     9 o ← texte() - un texte gravé
Lettres gravées : CEFGHINRS
```

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser() × 2    26 o sur place, dans main() + AdresseCase (10 o)
texte() × 2    22 o sur place, dans main() + EcrireTexte (10 o)
ecran() × 2    21 o sur place, dans main() + AttendreVBlank (16 o)
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : dessine deux fois le même carré de 8 × 8 pixels, écrit de deux
// façons, et en remplit deux lignes de l'écran.
//
// Ce qui est nouveau : la TUILE, un petit dessin de 8 × 8 pixels.
// Chaque texte entre guillemets est une RANGÉE de 8 pixels (8 rangées en tout).
// Chaque pixel a une des 4 nuances de la Game Boy d'origine :
// 0 = le plus clair  1 = clair  2 = sombre  3 = le plus sombre
// Deux bits par pixel suffisent : 4 valeurs possibles.
// poser(colonne, ligne, TUILE) pose la tuile dans une case de l'écran.

// Les quatre nuances, en chiffres : 0 le plus clair, 3 le plus sombre.
// Deux rangées de chaque nuance : 2 de 0, 2 de 1, 2 de 2, 2 de 3.

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <ecran>    // éteint ou rallume l'écran
#include <texte>    // écrit un texte à l'écran
#include <poser>    // pose une tuile sur une case du fond
```

## 2. UN DESSIN — CHIFFRES

```
Tuile CHIFFRES = {
    "00000000", "00000000",
    "11111111", "11111111",
    "22222222", "22222222",
    "33333333", "33333333",
};

// Exactement le même dessin, en signes : . - + #
// (. vaut 0, - vaut 1, + vaut 2, # vaut 3 : on choisit ce qui se lit le mieux.)
```



à l'écran

## 3. UN DESSIN — SIGNES

```
Tuile SIGNES = {
    ".....", ".....",
    "-----", "-----",
    "++++++", "++++++",
    "#####", "#####",
};
```



à l'écran

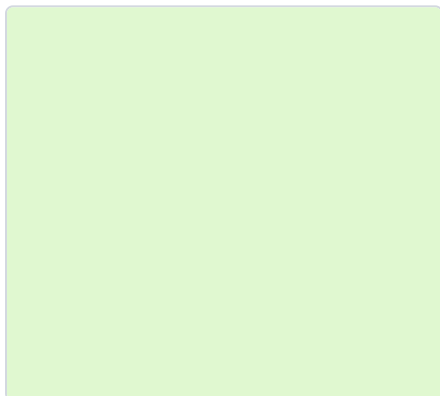
#### 4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    ecran(0);                               // Dessin écran éteint.
    texte(1, 1, "EN CHIFFRES");
    texte(1, 8, "EN SIGNES");
    for (uint8_t c = 0; c < 20; c++) { // Les 20 colonnes :
        poser(c, 3, CHIFFRES);           // une tuile CHIFFRES sur la ligne 3,
        poser(c, 10, SIGNES);            // une tuile SIGNES sur la ligne 10.
    }                                     // Les deux bandes sont identiques.
    ecran(1);                             // On rallume.

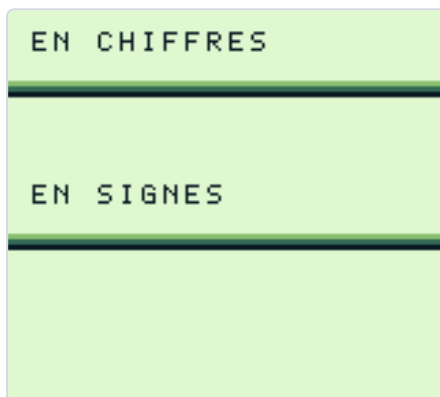
    while (true) {                          // La boucle du jeu.
        image();
    }
}
```

### 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

### 4. Ce qu'on doit voir

Deux bandes identiques de quatre nuances, du clair en haut au sombre en bas.

### 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les chiffres donnent les nuances 0, 1, 2, 3 (0123)
- ✓ les signes donnent exactement les mêmes (0123)
- ✓ la console est en mode d'origine, sans couleur

Ensuite : **96. Dessiner avec quatre nuances : lumière et ombre** — chapitre 15 : Les quatre nuances : la Game Boy d'origine. · gameboy3 — écrire une cartouche Game Boy en C++